

DEVOPS MASTER COURSE

Volume 11B

Prometheus Engineering, PromQL, Exporters and Alertmanager

Architecture • Scraping • Instrumentation • Exporters • PromQL • Rules • Alert Routing • High Availability

Chapters 11-20 | 2026 Edition

Course Standard Build, test, fail, diagnose, recover, document, and explain every component. Configuration alone is not completion.

Beginner to Production-Ready DevOps and SRE Engineering

About This Part

Volume 11B converts the observability requirements from Part A into an operational Prometheus monitoring platform. The part progresses from architecture and metric exposition through discovery, exporters, PromQL, recording rules, alert design, Alertmanager routing, high availability, and a production-style capstone.

Prometheus is treated as a reliability system rather than a graphing utility. Every chapter therefore connects collection behavior, query semantics, failure modes, resource consumption, security boundaries, ownership, testing, and operational evidence.

Volume 11 Roadmap

Part	Primary Scope	Status
11A	Observability foundations, Linux monitoring, telemetry, SLIs/SLOs	Completed
11B	Prometheus, PromQL, exporters, rules and Alertmanager	This book
11C	Grafana, Loki, Elastic/OpenSearch and centralized logging	Next
11D	OpenTelemetry, tracing, APM, profiling and eBPF	Planned
11E	SRE operations, incidents, capacity, reliability and governance	Planned
11F	Enterprise projects, certifications, interviews and capstone	Planned

Scope, Safety and Cost Controls

- Run scrape-failure, alert-routing, load, cardinality, retention, network, and high-availability tests only in disposable or explicitly approved environments.
- Use synthetic targets, labels, credentials, notifications, and incident channels. Never use production paging routes for learning exercises.
- Define retention, storage capacity, sample limits, label limits, target limits, query limits, and cleanup before generating telemetry at scale.
- Treat metric labels and annotations as potentially sensitive data. Do not expose secrets, personal identifiers, tokens, full URLs with credentials, or customer-controlled free text.
- Validate configuration and rules before reload. Preserve a known-good configuration and a documented rollback path.
- Confirm current image tags, flags, compatibility, and feature maturity in official documentation before production adoption.

Part B Learning Outcomes

- Design a Prometheus architecture with clear scrape, rule, query, storage, alerting, availability, and security boundaries.
- Instrument applications and exporters with stable metric names, bounded labels, suitable metric types, and meaningful help text.
- Build static and discovered scrape jobs, then control target and sample labels through relabeling.
- Write correct PromQL for rates, ratios, aggregation, vector matching, quantiles, absence, change detection, saturation, and SLO measurements.
- Use recording rules to stabilize repeated queries and alerting rules to detect actionable, sustained conditions.
- Configure Alertmanager routing, grouping, inhibition, silences, templates, receivers, and high availability.
- Validate a small production-style monitoring platform under normal operation, failure, recovery, and configuration rollback.

Table of Contents

- Chapter 11 Prometheus Architecture and Deployment Fundamentals
- Chapter 12 Metric Exposition and Application Instrumentation
- Chapter 13 Service Discovery, Scrape Configuration and Relabeling
- Chapter 14 Exporter Engineering, Node Exporter, Blackbox Exporter and Pushgateway
- Chapter 15 PromQL Foundations: Selection, Rates, Aggregation and Time
- Chapter 16 Advanced PromQL: Vector Matching, Histograms and SLO Queries
- Chapter 17 Recording Rules, Rule Testing and Query Performance
- Chapter 18 Alerting Rules and Actionable Alert Engineering
- Chapter 19 Alertmanager Routing, Inhibition, Silences, Templates and HA
- Chapter 20 Production Prometheus Monitoring Platform Capstone
- Part B Command and Configuration Reference
- Prometheus Engineering Completion Checklist
- Ten Additional Practice Projects
- 30-Day Prometheus Practice Plan
- Glossary
- Official Documentation Index

Chapter 11: Prometheus Architecture and Deployment Fundamentals

Designing the collection, storage, query, rule and alerting control plane

Learning Objectives

- Explain the Prometheus server data path from discovery and scraping to TSDB, rules, queries and notifications.
- Distinguish single-server, high-availability pair, federation and remote-storage architectures.
- Plan retention, capacity, persistence, resource requests, backups, security and failure boundaries.
- Validate configuration, health, readiness, target state, runtime state and self-monitoring.
- Create a reversible local deployment suitable for the remaining labs.

Core Concepts

Pull-based collection

Prometheus normally discovers or receives a list of targets and periodically scrapes their HTTP metric endpoints. A successful scrape produces timestamped samples plus internal scrape health metrics. Pulling makes target liveness, scrape duration, sample count, and errors visible to the collector.

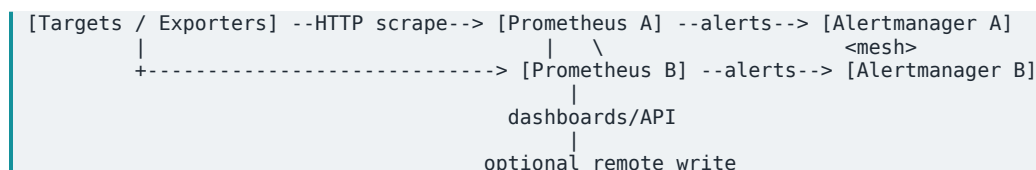
Local time-series database

The server stores recent samples in its local time-series database. Operational design must account for write-ahead logging, block compaction, retention, disk growth, I/O, memory, restart behavior, and the fact that local storage is not a general backup system.

Rules and alert delivery

The rule manager evaluates recording and alerting rules at configured intervals. Alerting rules produce labeled alert instances; Prometheus sends them to every configured Alertmanager endpoint. Alertmanager then groups, routes, inhibits, silences, and notifies.

Reference Architecture



Architecture Rule An HA pair improves availability but does not automatically create durable long-term storage or consistent global querying. Treat availability, durability and global-view requirements as separate design decisions.

Deployment Baseline

Illustrative Docker Compose service - pin and verify the image tag before use

```

services:
  prometheus:
    image: prom/prometheus:<approved-tag>
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --storage.tsdb.path=/prometheus
      - --storage.tsdb.retention.time=15d
      - --web.enable-lifecycle
    ports: ["9090:9090"]
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./rules:/etc/prometheus/rules:ro
  
```

```
- prometheus-data:/prometheus
volumes:
  prometheus-data: {}
```

Operational Endpoints and Checks

Check	Purpose	Example
Configuration syntax	Reject invalid YAML or unsupported fields	promtool check config prometheus.yml
Rule syntax	Reject invalid rule files	promtool check rules rules/*.yml
Health	Process is running	GET /-/healthy
Readiness	Server can serve traffic	GET /-/ready
Targets	Discovery and scrape status	Status > Targets or /api/v1/targets
Runtime	Flags, storage and build context	Status > Runtime & Build Information
Self-metrics	Capacity and failure signals	prometheus_* and process_* metrics

Production Readiness Checklist

- Pin approved versions and preserve checksums or signed provenance where required.
- Store configuration and rules in version control with review and automated validation.
- Use persistent storage with measured capacity and a tested restart/recovery procedure.
- Restrict administrative endpoints, reload capability, query access and network reachability.
- Monitor Prometheus scrape health, rule failures, TSDB health, query load, memory, CPU, disk and notification delivery.
- Document retention, recovery objectives, remote-storage behavior and ownership.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Prometheus starts then exits	Invalid config, file permissions or unsupported flag	Read startup logs and run promtool check config	Correct configuration; verify mounted paths and ownership
Target list is empty	Discovery or relabel rule removed targets	Inspect service discovery and dropped targets	Fix selectors/relabeling and reload safely
Disk grows unexpectedly	High ingestion, cardinality or retention	Inspect head series, samples, WAL and TSDB metrics	Reduce labels/samples, adjust retention, expand capacity
Queries time out	Expensive query, broad range or overload	Inspect query log/self-metrics and narrow expression	Add recording rules, limits or capacity; redesign query
Alerts absent from Alertmanager	Rules not firing or delivery path broken	Inspect Rules, Alerts, Alertmanagers and logs	Fix expression, labels, network, TLS or Alertmanager discovery

Hands-On Lab

1. Create a lab directory with configuration, rules, data and evidence subdirectories.
2. Run Prometheus using a pinned local container image or approved binary.
3. Validate configuration before startup.
4. Query health and readiness endpoints, then inspect runtime and build information.
5. Scrape Prometheus itself and confirm up, scrape duration and sample metrics.

6. Restart the server and confirm persistence.
7. Introduce one invalid configuration change, prove the validator rejects it, and restore the known-good file.
8. Record disk usage and core self-metrics before cleanup.

Validation Evidence

- Validated configuration output.
- Screenshot or API output showing the self-target is up.
- PromQL results for up and selected prometheus_* metrics.
- Restart evidence showing retained data.
- Failure-and-recovery log for the rejected configuration.
- Sanitized architecture diagram and capacity assumptions.

Interview Questions and Model Answers

Q: Why does Prometheus commonly use pull-based scraping?

A: It centralizes scrape scheduling and makes target availability, scrape latency and scrape errors observable while allowing the server to control collection cadence.

Q: Does two Prometheus replicas guarantee durable storage?

A: No. Replicas improve collector availability. Durability and long-term retention require separate storage and recovery design.

Q: What should be validated before a reload?

A: The main configuration, included scrape files, all rule files, secrets or mounted paths, target changes, and the rollback procedure.

Chapter Summary

- Prometheus combines discovery, scraping, local TSDB storage, PromQL, rule evaluation and alert forwarding.
- Availability, durability, retention and global querying are separate architecture concerns.
- Self-monitoring and safe configuration reloads are mandatory production capabilities.

Chapter 12: Metric Exposition and Application Instrumentation

Creating stable, useful and economically bounded time series

Learning Objectives

- Choose counters, gauges, histograms and summaries based on measurement semantics.
- Create metric and label names with clear namespace, unit and behavior.
- Instrument request, dependency, queue, worker and batch-job paths.
- Prevent label-cardinality explosions and sensitive-data leakage.
- Test exposition, concurrency, process restarts and metric correctness.

Metric Types and Semantics

Type	Use	PromQL Pattern	Common Error
Counter	Cumulative events that reset on restart	<code>rate(x_total[5m])</code> or <code>increase(x_total[1h])</code>	Graphing raw counter as activity
Gauge	Current value that can rise or fall	<code>avg_over_time(x[15m])</code>	Using a gauge for cumulative events
Histogram	Count observations in configured buckets	<code>histogram_quantile()</code> over bucket rates	Wrong buckets or missing aggregation by le
Summary	Client-calculated quantiles plus sum/count	<code>rate(sum())/rate(count)</code> for mean	Aggregating client quantiles across instances

Naming and Labels

- Use a stable application or subsystem prefix when it improves ownership and collision avoidance.
- Use base units such as seconds and bytes; include the unit in the metric name.
- Counters normally use the `_total` suffix.
- Labels should represent bounded dimensions that users will aggregate, filter or route on.
- Do not use request IDs, email addresses, usernames, raw paths, full error messages, timestamps, random values or unbounded customer input as labels.
- Keep one metric name semantically consistent across every target that exposes it.

Application Instrumentation Pattern

Illustrative Python instrumentation

```
from prometheus_client import Counter, Histogram, Gauge, start_http_server

REQUESTS = Counter(
    "checkout_http_requests_total",
    "Completed HTTP requests",
    ["method", "route", "status_class"],
)
LATENCY = Histogram(
    "checkout_http_request_duration_seconds",
    "HTTP request latency",
    ["method", "route"],
    buckets=(0.05, 0.1, 0.25, 0.5, 1, 2.5, 5),
)
INFLIGHT = Gauge("checkout_http_requests_in_flight", "In-flight HTTP requests")

start_http_server(8000)
```

Cardinality Budget Estimate series before deployment: metric families × unique label combinations × targets. Test the worst realistic label set, not only a happy-path sample.

Instrumentation by Workload

Workload	Essential Metrics	Operational Question
Online request service	request count, errors, duration, in-flight, dependency latency	Are users succeeding within the objective?
Queue worker	queue depth, oldest age, processing count/duration, retries, failures	Can workers keep up and recover?
Batch job	last success time, duration, processed records, failures	Did the job finish correctly and recently?
Cache	requests, hits/misses, evictions, capacity	Is the cache helping or causing pressure?
Database client	query count, duration, errors, pool usage	Is the dependency slow, failing or saturated?

Production Readiness Checklist

- Document metric purpose, type, unit, labels, owner and expected cardinality.
- Use route templates such as `/orders/{id}`, not raw request paths.
- Expose HELP and TYPE metadata and verify valid exposition.
- Make instrumentation thread-safe and ensure timers observe every exit path.
- Test counter resets, missing labels, exceptions and process restarts.
- Protect the metrics endpoint with appropriate network and authentication controls.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Series count grows rapidly	Unbounded or multiplicative labels	Count series by metric and label; inspect TSDB status	Remove or normalize labels; drop unsafe samples
Rate becomes negative or erratic	Wrong metric type or manual reset	Inspect raw samples and process restarts	Use a counter library and <code>rate()</code> over a suitable range
Latency quantile is empty	No traffic, wrong bucket query or missing le	Inspect <code>_bucket/_count/_sum</code> series	Fix instrumentation and aggregation
Metrics endpoint is slow	Expensive collection during scrape	Measure scrape duration and exporter self-metrics	Cache collection or move work out of scrape path
Sensitive value appears in labels	Raw user-controlled data was labeled	Inspect exposition and stored series	Remove label, delete/expire data per policy, add review tests

Hands-On Lab

1. Build a small HTTP service with request count, latency and in-flight metrics.
2. Use route templates and status classes as bounded labels.
3. Generate successful, failing, fast and slow synthetic requests.
4. Scrape the service and inspect raw exposition.
5. Write PromQL to calculate request rate, error ratio and mean duration.
6. Add a deliberately unsafe `request_id` label in a disposable branch, measure series growth, then remove it.
7. Restart the service and explain the counter-reset behavior.
8. Document the final metric contract.

Validation Evidence

- Source code and dependency manifest.
- Raw /metrics sample with HELP and TYPE lines.
- PromQL results for rate, errors and latency.
- Before/after cardinality counts.
- Metric contract table and review checklist.
- Restart test showing correct rate handling.

Interview Questions and Model Answers

Q: Why are raw URLs dangerous labels?

A: Dynamic path segments create unbounded label values and therefore an uncontrolled number of time series. Use normalized route templates.

Q: When should a histogram be preferred over a summary?

A: When server-side aggregation across instances and configurable bucket-based SLO analysis are required.

Q: How do you calculate average latency from a histogram?

A: Divide the rate of the `_sum` series by the rate of the `_count` series over the same range, grouped by the same dimensions.

Chapter Summary

- Metric type is defined by behavior, not by the desired chart shape.
- Bounded labels and stable naming are essential for correctness and cost control.
- Instrumentation must be validated under errors, concurrency, restarts and worst-case label combinations.

Chapter 13: Service Discovery, Scrape Configuration and Relabeling

Controlling which targets and samples enter the monitoring system

Learning Objectives

- Build global and per-job scrape settings with appropriate intervals and timeouts.
- Use static targets and understand the role of dynamic service-discovery mechanisms.
- Apply target relabeling, metric relabeling and alert relabeling at the correct stage.
- Normalize ownership, environment and service labels without creating conflicts.
- Debug discovery, dropped targets, scrape failures, duplicate series and sample limits.

Scrape Lifecycle

```
service discovery
-> discovered labels (__meta_*)
-> target relabeling
-> final target labels and scrape address/path/scheme
-> HTTP scrape
-> metric relabeling
-> ingestion into TSDB
-> rule evaluation
-> alert relabeling before Alertmanager
```

Baseline Configuration

```
global:
  scrape_interval: 30s
  scrape_timeout: 10s
  evaluation_interval: 30s

rule_files:
  - /etc/prometheus/rules/*.yaml

scrape_configs:
  - job_name: "checkout-api"
    metrics_path: /metrics
    static_configs:
      - targets: ["checkout:8000"]
        labels:
          environment: lab
          service: checkout
          team: payments
    sample_limit: 5000
    label_limit: 30
```

Relabeling Examples

```
relabel_configs:
  - source_labels: [__meta_kubernetes_namespace]
    regex: production
    action: keep
  - source_labels: [__meta_kubernetes_pod_label_app_kubernetes_io_name]
    target_label: service
  - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
    regex: (.+)
    target_label: __metrics_path__

metric_relabel_configs:
  - source_labels: [__name__]
    regex: "debug_.*"
    action: drop
```

Stage Matters Target relabeling changes or removes scrape targets before collection. Metric relabeling changes or drops scraped samples immediately before ingestion. Use the earliest correct stage and test the effect explicitly.

Scrape Design Decisions

Decision	Trade-off	Preferred Evidence
Scrape interval	Freshness versus target/network/storage cost	SLO detection needs and measured scrape cost
Timeout	Slow exporters versus blocked scrape slots	Observed scrape duration and failure behavior
Target labels	Query usability versus series multiplication	Label contract and cardinality estimate
Metric drops	Cost reduction versus diagnostic blind spots	Approved inventory and before/after sample count
Discovery scope	Automation versus accidental broad collection	Selectors, ownership and dropped-target review

Production Readiness Checklist

- Set scrape timeout below scrape interval and measure exporter duration.
- Use explicit job names, ownership labels and environment labels.
- Apply limits for samples, labels, label lengths and targets where supported and appropriate.
- Keep secrets in approved secret mechanisms, not committed YAML.
- Review dropped-target memory and service-discovery load.
- Test relabel rules with representative metadata before rollout.
- Monitor duplicate-series, out-of-order, scrape, discovery and ingestion errors.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Target is discovered but not active	Relabel keep/drop rule removed it	Inspect Service Discovery and dropped labels	Correct regex/source labels and reload
Target is down with connection refused	Wrong address/port or service not listening	curl from Prometheus network namespace	Fix target address, network policy or service
Context deadline exceeded	Exporter exceeds scrape timeout	Inspect scrape duration and exporter logs	Optimize exporter or adjust justified timeout
Duplicate time series error	Targets produce identical final labels	Inspect target labels and honor_labels behavior	Add stable distinguishing labels or fix exporter
High samples dropped	sample_limit or label limit exceeded	Inspect scrape error and sample counts	Reduce exposed metrics or adjust reviewed limit

Hands-On Lab

1. Add two static application targets and one node exporter target.
2. Assign service, team and environment labels.
3. Create one target relabel rule that retains only the lab environment.
4. Create one metric relabel rule that drops an intentionally generated debug metric.
5. Validate and reload the configuration.
6. Inspect active targets, dropped targets, labels and scrape metrics.

7. Break one port, observe failure, repair it and record recovery time.
8. Compare samples scraped before and after metric filtering.

Validation Evidence

- promtool validation output.
- Active and dropped target evidence.
- Before/after sample counts.
- Scrape-failure timeline and root cause.
- Final label contract.
- Sanitized configuration diff and rollback result.

Interview Questions and Model Answers

Q: What is the difference between `relabel_configs` and `metric_relabel_configs`?

A: Target relabeling operates on discovered target labels before scraping; metric relabeling operates on scraped samples just before ingestion.

Q: Why can `honor_labels` be risky?

A: It can allow target-provided labels to override server-side labels, potentially breaking ownership or uniqueness conventions.

Q: How do you debug a missing target?

A: Check discovery first, then relabeling and dropped targets, then final address/path/scheme, network reachability, endpoint response and scrape logs.

Chapter Summary

- Discovery defines candidates; relabeling defines the final targets and labels.
- Scrape intervals, timeouts and limits must be justified by measured reliability and cost needs.
- Target and sample filtering require evidence because an incorrect rule can silently remove critical telemetry.

Chapter 14: Exporter Engineering, Node Exporter, Blackbox Exporter and Pushgateway

Monitoring systems that do not expose application-native Prometheus metrics

Learning Objectives

- Explain the exporter pattern and when direct instrumentation is better.
- Collect Linux host metrics with node exporter and select collectors deliberately.
- Perform black-box HTTP, TCP, DNS and ICMP probes with a multi-target exporter.
- Use the Pushgateway only for suitable short-lived batch-job metrics.
- Design exporter self-metrics, timeouts, concurrency, security and failure handling.

Exporter Selection

Pattern	Best Fit	Key Risk
Direct instrumentation	Applications you own	Poor schema or uncontrolled labels
Node/system exporter	Host OS and kernel metrics	Collector overhead and mount/device noise
Black-box exporter	External user-visible reachability and protocol checks	Confusing probe success with service correctness
Protocol/database exporter	Third-party service with queryable status	Credential scope and expensive collection
Pushgateway	Service-level metrics from short-lived batch jobs	Stale groups and misuse for long-running services

Node Exporter Queries

```
# CPU busy ratio by instance
1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]))

# Available memory ratio
node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes

# Filesystem free ratio, excluding pseudo filesystems
node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs"}
/ node_filesystem_size_bytes{fstype!~"tmpfs|overlay|squashfs"}
```

Blackbox Multi-Target Pattern

```
- job_name: blackbox-http
  metrics_path: /probe
  params:
    module: [http_2xx]
  static_configs:
    - targets:
      - https://example.invalid/health
  relabel_configs:
    - source_labels: [__address__]
      target_label: __param_target
    - source_labels: [__param_target]
      target_label: instance
    - target_label: __address__
      replacement: blackbox-exporter:9115
```

Pushgateway Safety

- Use it for service-level metrics from short-lived jobs when Prometheus cannot scrape the process during execution.

- Do not attach machine or instance lifecycle metrics that should disappear automatically with a target.
- Choose a stable grouping key and delete pushed groups when they are no longer valid.
- Export completion time, success state, duration and processed work rather than every internal event.
- Monitor the Pushgateway itself and define stale-metric detection.

Production Readiness Checklist

- Run exporters with least privilege and only required collectors or queries.
- Bind endpoints to approved networks and add TLS/authentication at an appropriate layer.
- Set collection timeouts below the Prometheus scrape timeout.
- Expose exporter process, scrape and collection-error metrics.
- Normalize noisy filesystem, device, interface and mount labels.
- Document credential rotation and failure behavior for third-party exporters.
- Define cleanup ownership for Pushgateway groups.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
node exporter target down	Process, firewall, port or network namespace issue	curl endpoint from collector path	Restore process/network and verify service supervision
Filesystem alert on ephemeral mount	Mount filtering is incomplete	Inspect fstype, mountpoint and device labels	Exclude approved ephemeral/pseudo filesystems
probe_success=0 but exporter up	Target probe failed while exporter is healthy	Inspect probe phase and status metrics	Fix target, DNS, TLS, protocol or module settings
Exporter scrape times out	Backend query is slow or collectors block	Inspect exporter duration/error metrics	Limit collectors/queries and enforce timeout
Batch metric remains forever	Pushed group was not deleted	Inspect grouping labels and push time	Delete stale group and automate cleanup

Hands-On Lab

1. Deploy node exporter and scrape one Linux host.
2. Write CPU, memory, disk-space and network-throughput queries.
3. Deploy blackbox exporter and probe one healthy and one intentionally broken HTTP endpoint.
4. Inspect DNS, connect, TLS, processing and total probe duration.
5. Create a short-lived synthetic batch job that pushes last-success and duration metrics.
6. Delete the group after completion and verify disappearance.
7. Disable or break one exporter, observe failure, restore it and document the timeline.
8. Write a least-privilege and network-exposure review.

Validation Evidence

- Exporter configurations and service status.
- PromQL results for four host resources.
- Blackbox healthy/broken probe comparison.
- Push/delete evidence for the batch job.
- Scrape-duration and collection-error metrics.
- Failure and recovery timeline.

Interview Questions and Model Answers

Q: What is the difference between white-box and black-box monitoring?

A: White-box monitoring observes internal state exposed by the service; black-box monitoring tests externally visible behavior from a probe location.

Q: Why should Pushgateway not replace scraping for long-running services?

A: Pushed series are not automatically tied to process lifecycle, so stale metrics can remain and target health semantics are lost.

Q: What should a custom exporter expose about itself?

A: Collection duration, errors, last successful collection, process metrics, build information and bounded details about backend status.

Chapter Summary

- Prefer direct instrumentation for owned applications and exporters for external systems or protocol probes.
- Exporter health and probe target health are separate signals.
- Pushgateway use requires strict workload selection, grouping and cleanup discipline.

Chapter 15: PromQL Foundations: Selection, Rates, Aggregation and Time

Turning time-series samples into correct operational answers

Learning Objectives

- Distinguish instant vectors, range vectors, scalars and strings.
- Use label matchers, offsets, aggregation operators and core functions.
- Calculate counter rates, increases, ratios, averages and saturation safely.
- Choose range windows and query steps that match scrape behavior.
- Debug empty, duplicated, misleading and expensive results.

Core Data Shapes

Shape	Meaning	Example
Instant vector	One recent sample for each selected series	<code>up{job="checkout"}</code>
Range vector	A time window of samples per selected series	<code>http_requests_total[5m]</code>
Scalar	Single numeric value	<code>scalar(count(up))</code>
String	Rarely used literal string	<code>"lab"</code>

PromQL Building Blocks

```
# Selectors
up{job="checkout", environment=~"prod|staging"}

# Counter activity
sum by (service) (rate(http_requests_total[5m]))

# Error ratio
sum(rate(http_requests_total{status_class="5xx"}[5m]))
/
sum(rate(http_requests_total[5m]))

# Mean request duration from histogram sum/count
sum(rate(http_request_duration_seconds_sum[5m]))
/
sum(rate(http_request_duration_seconds_count[5m]))

# Compare with one hour earlier
sum(rate(http_requests_total[5m] offset 1h))
```

Counter Query Discipline

- Apply `rate()`, `irate()` or `increase()` to the raw counter series before aggregating across replicas so resets can be detected correctly.
- Use `rate()` for stable alerting and dashboards; reserve `irate()` for very short-term volatile inspection.
- Choose a range containing enough scrapes to tolerate jitter and missed collection.
- Use `increase()` when the operational question is event count over a fixed window.
- Compare numerator and denominator over identical windows and compatible label dimensions.

PromQL Review Question For every expression, state the operational question, input metric semantics, output unit, output labels, missing-data behavior and expected result during a process restart.

Aggregation

Expression	Result
sum by (service) (...)	Preserves service; removes other labels
sum without (instance, pod) (...)	Removes listed replica labels; preserves remaining context
topk(5, ...)	Five largest current samples per evaluation
count by (job) (up)	Number of current target series per job
quantile(0.95, gauge_metric)	Cross-series quantile of current gauge samples, not histogram latency

Production Readiness Checklist

- Document the output unit and labels for shared queries.
- Test queries with zero traffic, missing targets, counter resets and partial failure.
- Use identical windows for ratio components unless a deliberate reason is documented.
- Avoid broad regex and unbounded time ranges in frequent dashboard queries.
- Preserve routing/ownership dimensions needed downstream.
- Use query analysis and recording rules for repeated expensive expressions.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Query returns no data	Metric/labels absent, time outside data or matcher wrong	Inspect raw metric and label values	Correct selector/window and define missing-data behavior
Rate spikes after restart	Aggregation occurred before rate or bad source counter	Inspect per-instance raw counter	Apply rate per series; fix instrumentation
Ratio exceeds expected bounds	Numerator/denominator labels or windows differ	Inspect each side and output labels	Align dimensions and aggregate consistently
Many-to-many error	Binary operation has non-unique match keys	Count series by intended match labels	Fix on/ignoring/group modifiers or pre-aggregate
Dashboard is slow	Wide range, high cardinality or repeated heavy expression	Measure query duration and series touched	Narrow scope, add recording rule, reduce cardinality

Hands-On Lab

1. Use the lab application and exporters from earlier chapters.
2. List metric names and label values before writing derived queries.
3. Write queries for traffic, error ratio, mean latency, CPU busy ratio and filesystem free ratio.
4. Restart one application instance and verify rate behavior.
5. Stop one target and compare up, absent(), count and aggregation results.
6. Create one intentionally incorrect ratio with mismatched labels, explain it and correct it.
7. Run the same expression as instant and range queries and explain the difference.
8. Record output units and label sets for every final query.

Validation Evidence

- PromQL worksheet with question, expression, unit and labels.
- Screenshots or API JSON for correct results.

- Counter-reset test evidence.
- Missing-target comparison.
- Incorrect-versus-correct ratio analysis.
- Query timing for one broad and one optimized expression.

Interview Questions and Model Answers

Q: Why should `rate()` be applied before `sum()`?

A: `rate()` needs each original series to detect its individual counter resets; aggregation first can hide resets.

Q: What does `by()` do in an aggregation?

A: It preserves the listed labels and removes all other labels from the output grouping.

Q: Why can a five-minute rate be empty shortly after startup?

A: The range may not contain enough samples for the function to calculate a rate, especially with sparse or delayed scrapes.

Chapter Summary

- PromQL answers questions by transforming typed collections of labeled time series.
- Counter functions, windows, labels and aggregation order determine correctness.
- Every production query needs tested behavior for missing data, resets, partial failure and cost.

Chapter 16: Advanced PromQL: Vector Matching, Histograms and SLO Queries

Combining dimensions and deriving reliable latency and availability indicators

Learning Objectives

- Use one-to-one and many-to-one vector matching intentionally.
- Calculate histogram quantiles and bucket-based SLO compliance.
- Understand classic and native histogram query implications at a practical level.
- Build multi-window burn-rate and saturation expressions.
- Use subqueries and predictive/change functions only when justified.

Vector Matching

```
# CPU mode proportion per instance: many mode series divided by one total per instance
sum by (instance, mode) (rate(node_cpu_seconds_total[5m]))
/
ignoring(mode) group_left
sum by (instance) (rate(node_cpu_seconds_total[5m]))

# Enrich per-instance error ratio with owner info metric
instance_error_ratio
* on(instance) group_left(team, service)
target_owner_info
```

A binary vector operation must have a unique matching relationship. Use `on()` to match only selected labels or `ignoring()` to exclude selected labels. `group_left/group_right` explicitly permits many-to-one matching and can copy labels from the one-side; misuse can duplicate or misattribute data.

Classic Histogram Quantiles

```
histogram_quantile(
  0.95,
  sum by (le, service) (
    rate(http_request_duration_seconds_bucket[5m])
  )
)
```

For classic histograms, preserve the `le` label until `histogram_quantile()` is applied. Bucket boundaries determine the precision and usefulness of the estimated quantile. Quantiles are estimates, not exact request records.

Bucket-Based SLO Compliance

```
# Fraction of requests completed within 300 ms
sum(rate(http_request_duration_seconds_bucket{le="0.3"}[5m]))
/
sum(rate(http_request_duration_seconds_count[5m]))

# Error-budget burn rate for a 99.9% objective
(
  1 - (
    sum(rate(http_requests_total{status_class!="5xx"}[5m]))
    /
    sum(rate(http_requests_total[5m]))
  )
) / (1 - 0.999)
```

Advanced Function Review

Function/Pattern	Use	Caution
<code>absent()</code> / <code>absent_over_time()</code>	Detect no matching series	Must distinguish expected absence from failure
<code>changes()</code> / <code>resets()</code>	Detect value changes or counter resets	Requires suitable window and metric

Function/Pattern	Use	Caution
		semantics
predict_linear()	Simple linear projection for gauges	Not a capacity plan; unstable on non-linear data
deriv()	Trend of gauge samples	Not for counters
subquery expr[range:resolution]	Evaluate an expression across a range	Can be expensive and hard to reason about
clamp_min/max	Bound displayed/calculated values	May hide instrumentation errors if overused

Production Readiness Checklist

- Prove match-key uniqueness before adding group_left or group_right.
- Preserve required histogram labels and use consistent bucket schemas across aggregated targets.
- Test SLO queries against known synthetic traffic and hand calculations.
- Define behavior when denominator is zero or series are missing.
- Record quantile error limitations and bucket rationale.
- Precompute repeated burn-rate and histogram expressions when appropriate.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Many-to-many matching error	Both sides contain multiple series for match key	count by(match labels) on each input	Pre-aggregate or correct match labels
Quantile returns NaN/empty	No observations or missing bucket data	Inspect count and bucket rates	Generate traffic; fix scrape/instrumentation
Quantile is implausible	Bad bucket layout or inconsistent aggregation	Inspect bucket boundaries and cumulative rates	Redesign buckets and preserve le
SLO ratio is NaN	Zero denominator	Inspect traffic count and missing series	Define explicit no-traffic behavior
Burn rate is wrong by dimension	Good/total aggregations do not align	Compare output labels on both sides	Use identical aggregation and filters

Hands-On Lab

1. Generate synthetic latency in three bands around the chosen SLO threshold.
2. Calculate the histogram quantile and bucket-based compliance ratio.
3. Compare the estimated p95 with the known generated distribution.
4. Build one valid many-to-one join with an info metric.
5. Create a duplicate info series, reproduce the matching error and fix uniqueness.
6. Compute a 5-minute and 1-hour error-budget burn rate for a 99.9% objective.
7. Stop traffic and document zero-denominator behavior.
8. Create recording-rule candidates for the final SLO queries.

Validation Evidence

- Synthetic workload definition.
- Bucket and count query output.

- Quantile-versus-known-distribution comparison.
- Vector-matching failure and repair evidence.
- Burn-rate calculations with hand verification.
- Documented no-traffic behavior.

Interview Questions and Model Answers

Q: Why is the `le` label required for classic histogram quantiles?

A: It identifies cumulative bucket boundaries; `histogram_quantile()` needs those boundaries to estimate the quantile.

Q: What causes a many-to-many matching error?

A: More than one series exists on both sides for the selected match labels, so Prometheus cannot determine a unique pairing.

Q: Why are bucket-based SLO ratios often preferable to p95 alerts?

A: They directly measure the fraction of events meeting a defined threshold and align with an availability-style objective; a percentile alone does not state how much traffic violated the threshold.

Chapter Summary

- Vector matching is a data-model operation and requires proven uniqueness.
- Histogram quantiles depend on bucket design and correct aggregation.
- SLO queries must be validated with known traffic, zero-volume conditions and consistent dimensions.

Chapter 17: Recording Rules, Rule Testing and Query Performance

Precomputing stable operational metrics without hiding mistakes

Learning Objectives

- Create recording-rule groups with clear names, intervals and ownership.
- Use naming conventions that reveal aggregation level and operation.
- Validate rule syntax and test rule results with deterministic input series.
- Measure rule evaluation duration, failures and output cardinality.
- Decide when to optimize queries, add rules or redesign source metrics.

Recording Rule Example

```
groups:
- name: checkout-sli.rules
  interval: 30s
  rules:
  - record: service:http_requests:rate5m
    expr: sum by (service) (rate(http_requests_total[5m]))
  - record: service:http_errors:ratio_rate5m
    expr: |
      sum by (service) (rate(http_requests_total{status_class="5xx"}[5m]))
      /
      sum by (service) (rate(http_requests_total[5m]))
```

Why Rules Help

- Repeated dashboards and alerts query a compact, consistent result instead of recalculating an expensive expression.
- Rules create a reviewed semantic interface between raw telemetry and consumers.
- Rules can preserve stable labels and units across dashboards, alerts and reports.
- Rules do not repair incorrect source metrics or expressions; they can make an error more widely consumed.

Unit Testing Rules

Illustrative promtool test structure; verify exact expected values

```
rule_files:
- checkout.rules.yml

evaluation_interval: 1m

tests:
- interval: 1m
  input_series:
  - series: 'http_requests_total{service="checkout",status_class="2xx"}'
    values: '0 60 120 180 240 300'
  - series: 'http_requests_total{service="checkout",status_class="5xx"}'
    values: '0 0 3 3 6 6'
  promql_expr_test:
  - expr: service:http_errors:ratio_rate5m
    eval_time: 5m
    exp_samples:
    - labels: '{service="checkout"}'
      value: 0.02
```

Query and Rule Performance

Signal	Meaning	Response
Rule evaluation failures	Expression or execution failure	Inspect logs, expression and input data
Evaluation duration near interval	Rule group may fall behind	Optimize, split groups or add capacity

Signal	Meaning	Response
Large output series count	Rule preserves too many dimensions	Reduce labels or redesign aggregation
Dashboard repeated heavy query	Cost multiplied by refresh/users	Create reviewed recording rule
High query concurrency/latency	Workload exceeds capacity or query is inefficient	Limit, optimize, cache externally or scale architecture

Production Readiness Checklist

- Validate configuration and rules in CI before merge and deployment.
- Store rule ownership, purpose, unit, output labels and consumers.
- Use deterministic unit tests for normal, failure, missing-data and reset cases.
- Monitor rule evaluation duration, failures, missed iterations and output series.
- Keep evaluation intervals aligned with source scrape interval and detection needs.
- Version and roll back rule changes independently where possible.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Rule file rejected	YAML or PromQL syntax error	promtool check rules and server logs	Fix syntax; add CI validation
Rule produces no series	Input selector absent or label mismatch	Run expression and inspect raw inputs	Correct selectors/window; define missing behavior
Rule output explodes	Unwanted labels preserved	count output by labels and inspect expression	Aggregate away unnecessary dimensions
Evaluation takes too long	Heavy query, high cardinality or overloaded server	Inspect rule duration and query plan evidence	Optimize expression, split group, add rule/capacity
Tests disagree with production	Test fixtures omit resets/missing/jitter	Compare real input shape and labels	Improve fixtures and include edge cases

Hands-On Lab

1. Create recording rules for service request rate, error ratio and p95 latency.
2. Validate the rule file with promtool.
3. Create deterministic test input for normal traffic, errors and a counter reset.
4. Run rule tests and correct expected values until they match hand calculations.
5. Load rules and inspect group health, evaluation time and output labels.
6. Create a deliberately high-cardinality rule, measure output, then redesign it.
7. Compare dashboard query duration before and after using recording rules.
8. Commit rules, tests and documentation as one reviewed unit.

Validation Evidence

- Rule and test files.
- Validation and test output.
- Hand calculation worksheet.
- Rule group health and evaluation metrics.
- Before/after output series count.
- Before/after query duration.
- Versioned rollback evidence.

Interview Questions and Model Answers

Q: When should you create a recording rule?

A: When a correct expression is expensive or repeatedly evaluated, or when a reviewed stable semantic metric is needed by multiple consumers.

Q: Why test recording rules?

A: Syntax validation does not prove semantics. Deterministic input and expected output catch label, reset, missing-data and arithmetic errors.

Q: What happens if rule evaluation duration approaches the interval?

A: The group may not complete reliably before its next scheduled evaluation, increasing delay and load; the query or architecture must be corrected.

Chapter Summary

- Recording rules are reviewed semantic products, not only performance shortcuts.
- Rule tests must include edge cases and verify exact labels and values.
- Monitor rule health, duration and output cardinality continuously.

VOLUME 11B | CHAPTER 18

Chapter 18: Alerting Rules and Actionable Alert Engineering

Detecting conditions that require a specific human or automated response

Learning Objectives

- Distinguish alert conditions, notifications, incidents and tickets.
- Create alerts from user impact, SLO burn, saturation and platform failure signals.
- Use `for` and `keep_firing_for` deliberately.
- Design stable labels and useful annotations with runbooks and diagnostic context.
- Test pending, firing, recovery, missing-data and notification behavior.

Alert Rule Example

```
groups:
- name: checkout-alerts
  rules:
  - alert: CheckoutHighErrorBudgetBurn
    expr: service:http_errors:ratio_rate5m / (1 - 0.999) > 14.4
    for: 5m
    keep_firing_for: 5m
    labels:
      severity: page
      service: checkout
      team: payments
    annotations:
      summary: "Checkout is rapidly consuming its error budget"
      description: "5m burn rate is {{ $value | printf \"%.2f\" }}."
      runbook_url: "https://runbooks.example.invalid/checkout/high-burn"
```

Actionability Contract

Field	Question It Must Answer
Condition	What measurable state requires action?
Owner	Which team is responsible now?
Severity	What response time and interruption level apply?
Impact	Which user journey or objective is at risk?
Annotations	What happened, where, and what is the current value?
Runbook	What should the responder verify and do first?
Recovery	How will the alert resolve and how is recovery confirmed?

Alert Categories

- Symptom alerts: user-visible failures, latency, availability or error-budget burn.
- Cause alerts: resource, dependency or component faults; usually lower priority unless directly actionable.
- Missing telemetry alerts: monitoring blind spots that can invalidate other conclusions.
- Monitoring-system alerts: Prometheus, rules, Alertmanager and notification paths.
- Capacity alerts: forecasted exhaustion with enough lead time for a planned response.
- Security or audit alerts: often require separate ownership, routing, retention and tools.

Paging Standard A page should represent urgent user or service risk with an immediate, documented response. Non-urgent work belongs in tickets, reports or planned reviews.

Alert Testing

- Unit-test expressions and final alert labels/annotations with promtool fixtures.
- Generate a controlled condition and observe inactive, pending, firing and resolved states.
- Confirm the notification reaches a synthetic receiver with the expected group and route.
- Test missing data and a dead Prometheus target separately from a healthy zero value.
- Confirm the runbook works from a clean responder context.
- Measure alert detection, notification and recovery latency.

Production Readiness Checklist

- Every paging alert has an owner, severity definition, impact statement and tested runbook.
- Prefer SLO/user-impact alerts over brittle implementation-threshold pages.
- Keep labels stable and bounded; place changing values in annotations.
- Use for only when sustained duration is part of the operational requirement.
- Test rules and routes before production rollout.
- Review page volume, duplicates, false positives, false negatives, time-to-acknowledge and post-incident usefulness.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Alert never becomes firing	Expression false, for not elapsed or missing data	Inspect Rules/Alerts and run expression at evaluation times	Fix expression/window or generate valid test condition
Alert flaps	Threshold near noise, no duration or unstable data	Inspect time series around transitions	Use SLO-based signal, hysteresis pattern or justified duration
Too many alerts per outage	Alert labels preserve instance-level dimensions	Inspect alert fingerprints and grouping labels	Aggregate by service/impact or improve Alertmanager grouping
Wrong team receives alert	Ownership label or route matcher wrong	Inspect final labels and routing tree	Fix label contract and route tests
Alert resolves before responder sees recovery	Transient data or notification timing	Inspect alert state and keep_firing_for behavior	Correct signal/window and use justified keep_firing_for

Hands-On Lab

1. Create one page-level burn-rate alert and one ticket-level disk-capacity alert.
2. Add stable ownership labels and useful annotations.
3. Write promtool tests for inactive, pending/firing and missing-data cases.
4. Load the rules and observe state transitions under synthetic failures.
5. Send alerts to a local webhook receiver and inspect payloads.
6. Verify grouping and recovery notifications.
7. Run the linked runbook from a clean shell and record gaps.
8. Conduct a small alert review: action, urgency, owner, context and outcome.

Validation Evidence

- Alert rule and test files.
- State-transition timeline.
- Webhook payload with sanitized labels/annotations.
- Detection and notification latency.
- Runbook execution record.
- Alert review decision and improvements.

Interview Questions and Model Answers

Q: What is the difference between an alert and a notification?

A: An alert is a labeled state produced by a rule; a notification is a message generated after Alertmanager grouping, routing, inhibition and silence processing.

Q: What does the for clause do?

A: It requires the expression to remain active for the specified duration before the alert transitions from pending to firing.

Q: Why should changing values usually be annotations instead of labels?

A: Labels define alert identity and routing. Changing them creates new alert fingerprints, breaks grouping and can increase noise.

Chapter Summary

- Actionability, ownership and tested response are the definition of a useful alert.
- Alert states and notification behavior must be tested end to end.
- Stable labels, SLO-aligned signals and continuous quality review reduce noise.

Chapter 19: Alertmanager Routing, Inhibition, Silences, Templates and High Availability

Delivering the right grouped notification to the right responder

Learning Objectives

- Build a routing tree with matchers, grouping and timing parameters.
- Use inhibition to suppress dependent symptoms and silences for bounded maintenance.
- Create receiver integrations and templates without leaking secrets.
- Operate Alertmanager in a high-availability cluster and understand deduplication expectations.
- Test routes, notification payloads, reload, failure and recovery.

Routing Tree Example

```
route:
  receiver: default-ticket
  group_by: [alertname, service, environment]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - receiver: payments-page
      matchers:
        - team="payments"
        - severity="page"
      continue: false
    - receiver: platform-ticket
      matchers:
        - team="platform"

inhibit_rules:
  - source_matchers: [alertname="ClusterUnavailable"]
    target_matchers: [severity=~"page|ticket"]
    equal: [cluster, environment]
```

Grouping and Timing

Setting	Purpose	Failure Mode if Misused
group_by	Defines which alerts share one notification	Over-grouping hides scope; under-grouping creates storms
group_wait	Allows related alerts to arrive before first notification	Too long delays response; too short fragments groups
group_interval	Minimum time before notifying new alerts in an existing group	Too long hides new impact; too short creates noise
repeat_interval	Reminder cadence for unresolved groups	Too short annoys; too long loses visibility

Silences and Inhibition

- A silence matches alert labels for a bounded period and should include owner, reason, ticket/change reference and expiration.
- Inhibition automatically suppresses target alerts when a source alert is active and the configured equal labels match.
- Neither silences nor inhibition fixes a noisy rule. Review the rule and service design after repeated suppression.
- Test that a parent alert cannot inhibit unrelated environments, services or clusters.

High Availability

In an Alertmanager cluster, peers exchange notification and silence state for deduplication and coordination. Prometheus servers should send alerts to all intended Alertmanager instances directly rather than relying on a load balancer to select only one. Network partitions, peer reachability and receiver behavior must be tested.

Configuration Validation and Reload

```
amtool check-config alertmanager.yml
curl -X POST http://alertmanager:9093/-/reload
amtool alert query
amtool silence query
```

Production Readiness Checklist

- Store routing configuration and templates in version control and validate in CI.
- Use secret files or approved secret managers for receiver credentials.
- Define route ownership, fallback receiver and dead-letter/escalation behavior.
- Run at least two Alertmanager instances for required availability and monitor cluster health.
- Send alerts from each Prometheus replica to every Alertmanager replica.
- Audit silences, expiration, creators and reasons.
- Test receiver failure, peer loss, configuration rollback and notification deduplication.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
No notification received	Alert not firing, route mismatch, silence/inhibition or receiver failure	Inspect alert labels, status UI, route, silences and logs	Correct labels/route or receiver; remove invalid suppression
Duplicate notifications	HA peers not clustering or Prometheus/receiver retries	Inspect cluster status, peer network and notification logs	Fix peer mesh and consistent external URLs/config
Wrong alerts grouped together	group_by omits service/environment dimension	Inspect notification group labels	Adjust grouping based on response workflow
Silence does not match	Matcher syntax or label value differs	Use amtool and inspect exact alert labels	Correct bounded silence matcher
Config reload rejected	Invalid YAML, matcher, template or receiver field	Run amtool check-config and inspect logs	Fix config; retain known-good active state

Hands-On Lab

1. Deploy two Alertmanager instances in a lab mesh.
2. Configure Prometheus to send alerts to both instances.
3. Create page and ticket receivers using local webhook endpoints.
4. Build routes for payments and platform teams with a fallback receiver.
5. Trigger several related alerts and verify grouping.
6. Create an inhibition rule for a parent outage and prove unrelated alerts remain visible.
7. Create a short maintenance silence with reason and expiry, then verify it expires.
8. Stop one Alertmanager, verify notification continuity and restore it.
9. Break a receiver, observe retry/failure evidence, repair it and confirm delivery.

Validation Evidence

- Validated Alertmanager configuration.
- Routing decision matrix and test results.
- Grouped webhook payloads.

- Inhibition positive and negative tests.
- Silence creation, audit and expiry evidence.
- Peer-loss and recovery timeline.
- Receiver failure and retry evidence.

Interview Questions and Model Answers

Q: What is the purpose of Alertmanager grouping?

A: It combines related alert instances into a notification that reflects one operational event, reducing notification storms while preserving needed scope.

Q: What is the difference between inhibition and a silence?

A: Inhibition is automatic suppression based on another active alert and equal labels; a silence is a time-bounded matcher created for a known maintenance or exception.

Q: Why send alerts to all Alertmanager replicas?

A: It avoids making a load balancer a single selection point and allows the Alertmanager cluster to deduplicate and coordinate notifications.

Chapter Summary

- Alertmanager is the notification policy layer: grouping, routing, inhibition, silencing and receiver delivery.
- Configuration must be validated and tested with real label sets and synthetic receivers.
- High availability requires peer health, direct delivery to replicas and failure testing.

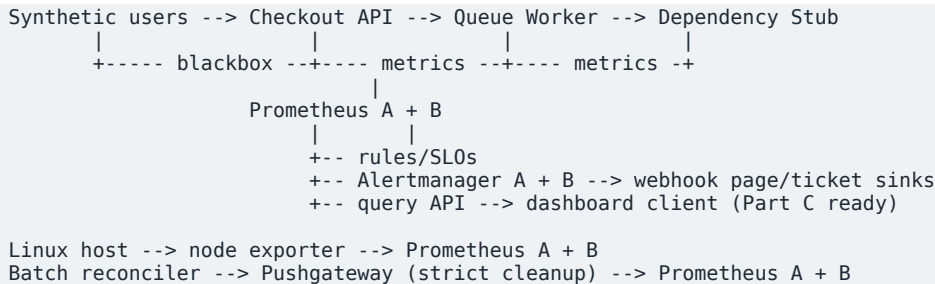
Chapter 20: Production Prometheus Monitoring Platform Capstone

Integrating instrumentation, scraping, rules, alerts, HA and operational evidence

Capstone Mission

Build a small production-style monitoring platform for a synthetic checkout service. The platform must monitor application behavior, Linux resources, external reachability and the monitoring system itself. It must survive controlled component failures, route actionable alerts, and produce an auditable evidence package.

Required Architecture



Functional Requirements

- Application metrics: traffic, status class, latency histogram, in-flight work, queue depth, worker outcomes, dependency latency and build information.
- Infrastructure metrics: CPU, memory, filesystem, network and process health with noisy pseudo resources filtered.
- Black-box metrics: external health, DNS/connect/TLS/processing duration and probe success.
- Batch metrics: last success, duration and outcome with explicit Pushgateway deletion.
- Recording rules: service traffic, error ratio, p95 latency, SLO compliance and error-budget burn.
- Alerts: fast burn page, slow burn ticket, instance/target loss, monitoring pipeline health and capacity warning.
- Alertmanager: team/severity routes, grouping, one inhibition rule, bounded silence workflow, local webhook receivers and two-node HA.
- Operations: validation in CI, safe reload, known-good rollback, backups/config preservation, retention plan and self-monitoring.

Repository Structure

```

volume-11b-capstone/
  app/                # synthetic service and workload
  compose/            # local platform deployment
  prometheus/
    prometheus.yml
  scrape/
    rules/
    tests/
  alertmanager/
    alertmanager.yml
    templates/
  exporters/
    blackbox.yml
  scripts/
    validate.sh
    smoke-test.sh
    failure-drills.sh
    cleanup.sh
  runbooks/
  evidence/
  README.md

```

Quality Gates

Gate	Acceptance Evidence
Configuration	promtool and amtool validation pass; invalid fixture fails
Targets	All expected targets up with correct labels; exclusions documented
Metrics	Metric contract, bounded labels and cardinality report
Queries	Hand-verified traffic, error, latency and SLO expressions
Rules	Deterministic rule tests for normal, failure, reset and missing data
Notifications	Correct route, group, inhibition, silence, recovery and fallback
Availability	Single Prometheus or Alertmanager failure does not eliminate monitoring/notification
Recovery	Known-good rollback and data/config restart procedures tested
Security	Network exposure, credentials, secrets and sensitive-label review
Operations	Runbooks, ownership, capacity, retention, monitoring and cleanup complete

Failure-Drill Matrix

Drill	Expected Detection	Expected Response/Recovery
Application returns 5xx	Error ratio and burn alerts	Runbook identifies release/dependency; restore service
Application stops	up=0 plus blackbox failure	Differentiate process and user-path failure; restart
Exporter hangs	Scrape timeout and exporter gap	Inspect collector duration; disable/repair collector
Prometheus A stops	Replica B remains queryable and sends alerts	Restore A and verify target/rule state
Alertmanager A stops	B continues notification	Restore peer and confirm mesh
Receiver fails	Delivery failures/retries visible	Repair receiver and confirm eventual delivery
Cardinality injection	Head series/sample growth visible	Remove unsafe label and apply guardrail
Invalid config	Validation/reload rejects change	Restore known-good commit and prove active config unchanged

Capstone Execution

1. Write the architecture decision record, ownership map, SLO, capacity assumptions and threat/sensitive-data review.
2. Build the synthetic application and instrument every required workload path.

3. Deploy two Prometheus servers, exporters, Pushgateway, two Alertmanagers and synthetic webhook receivers.
4. Create discovery, relabeling, limits, rule files and deterministic tests.
5. Generate normal traffic and capture a clean baseline of targets, series, queries, rule health and notifications.
6. Run every failure drill one at a time. Capture detection, diagnosis, action, recovery and gaps.
7. Run validation and smoke tests from a clean environment or rebuilt stack.
8. Review alerts for urgency, ownership, context and actionability; remove or downgrade weak alerts.
9. Produce a sanitized portfolio report with diagrams, configuration excerpts, results, limitations and future improvements.
10. Execute cleanup and prove lab receivers, silences, pushed metrics, containers, volumes and test credentials are removed.

Capstone Scoring Rubric

Area	Weight	Full-Credit Standard
Architecture and requirements	15%	Clear boundaries, ownership, SLOs, assumptions and decisions
Instrumentation and cardinality	15%	Correct types, bounded labels, measured series and sensitive-data safety
Scraping and exporters	15%	Reliable targets, relabeling, limits and exporter self-monitoring
PromQL and rules	20%	Correct, tested, efficient and documented derived metrics
Alerting and Alertmanager	15%	Actionable alerts, tested routes, grouping, inhibition, silences and HA
Failure recovery	10%	Measured drills with diagnosis, recovery and rollback
Documentation and operations	10%	Runbooks, evidence, capacity, security, cleanup and limitations

Production Readiness Checklist

- All configurations and rules pass automated validation and deterministic tests.
- Every target, metric, rule, alert, receiver and silence has an owner and purpose.
- The platform monitors itself and reports collection, rule, storage, query and notification health.
- Capacity and cardinality are measured, not guessed.
- Failure drills are isolated, reversible and documented.
- Secrets and sensitive values are absent from repositories, labels, screenshots and portfolio artifacts.
- Cleanup and rollback procedures are verified.

Troubleshooting Matrix

Symptom	Likely Cause	Verification	Corrective Action
Baseline differs after rebuild	Unpinned dependencies or non-deterministic config	Compare image digests, files and startup logs	Pin dependencies and automate bootstrap
HA replicas show different targets	Discovery/relabel/config drift	Compare active config and target APIs	Use identical reviewed config and external labels only where needed
Alert storm during drill	Instance alerts not grouped or	Inspect fingerprints and route	Aggregate alerts, improve

Symptom	Likely Cause	Verification	Corrective Action
	inhibited	grouping	grouping/inhibition and retest
SLO rule disagrees with workload	Metric filters, labels or bucket threshold wrong	Hand-calculate from generated request log	Correct metric contract and rule tests
Recovery not visible	Alert/metric missing recovery semantics	Inspect state transitions and data gaps	Add recovery checks and resolve notification

Hands-On Lab

1. Complete the repository and quality gates listed above.
2. Run the entire stack from a clean checkout.
3. Execute the baseline smoke test.
4. Execute all eight failure drills.
5. Perform one peer review using the scoring rubric.
6. Correct every critical or high-severity issue and rerun affected tests.
7. Generate the final evidence index and portfolio summary.
8. Execute cleanup and archive only sanitized configuration, tests and evidence.

Validation Evidence

- Architecture and data-flow diagram.
- Validated configuration and rule-test reports.
- Target and cardinality inventory.
- PromQL worksheet with hand verification.
- Alert routing/inhibition/silence test matrix.
- Failure-drill timelines and recovery proof.
- Security/cost/capacity review.
- Final score, limitations and improvement backlog.

Interview Questions and Model Answers

Q: How would you prove a Prometheus platform is production-ready?

A: By validating configuration and rule semantics, measuring capacity and cardinality, monitoring the platform itself, testing target/rule/notification paths, performing component-failure and rollback drills, and documenting ownership and runbooks.

Q: What is the most common root cause of Prometheus cost and performance problems?

A: Uncontrolled time-series cardinality, often caused by unbounded or multiplicative labels, combined with expensive queries and unsuitable retention.

Q: How do HA Prometheus and HA Alertmanager differ?

A: Prometheus replicas independently scrape and evaluate rules; Alertmanager peers coordinate state and deduplicate notifications. Neither replaces a durability or global-query design.

Chapter Summary

- The capstone integrates every layer from metric contract to incident notification.
- Production readiness is demonstrated through evidence under normal, failure and recovery states.
- A monitoring platform is complete only when it is operable, secure, economical, testable and explainable.

Part B Command and Configuration Reference

Task	Command / Pattern
Validate Prometheus config	<code>promtool check config prometheus.yml</code>
Validate rule files	<code>promtool check rules rules/*.yml</code>
Run rule tests	<code>promtool test rules tests/*.test.yml</code>
Validate Alertmanager config	<code>amtool check-config alertmanager.yml</code>
Reload Prometheus	<code>POST /-/reload</code> when lifecycle reload is enabled
Reload Alertmanager	<code>POST /-/reload</code> or supported signal
Inspect targets	<code>GET /api/v1/targets</code>
Instant query API	<code>GET /api/v1/query?query=<expression></code>
Range query API	<code>GET /api/v1/query_range?query=<expression>&start=...&end=...&step=...</code>
Inspect alerts	<code>GET /api/v1/alerts</code> and Alertmanager UI/API
Inspect Alertmanager alerts	<code>amtool alert query</code>
Inspect silences	<code>amtool silence query</code>
Create silence	<code>amtool silence add <matchers> --duration=<d> --comment=<reason></code>
Delete Pushgateway group	<code>DELETE</code> the exact grouping-key URL after job completion

Core PromQL Patterns

```
# Availability of targets
avg(up{job="checkout"})

# Traffic
sum by (service) (rate(http_requests_total[5m]))

# Error ratio
sum by (service) (rate(http_requests_total{status_class="5xx"}[5m]))
/
sum by (service) (rate(http_requests_total[5m]))

# Mean latency
sum by (service) (rate(http_request_duration_seconds_sum[5m]))
/
sum by (service) (rate(http_request_duration_seconds_count[5m]))

# p95 latency from classic histogram
histogram_quantile(0.95,
  sum by (le, service) (rate(http_request_duration_seconds_bucket[5m])))

# CPU busy ratio
1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]))

# Missing expected series
absent(up{job="checkout"})
```

Prometheus Engineering Completion Checklist

- I can explain the complete scrape-to-notification data path and its failure boundaries.

- I can deploy Prometheus with persistent storage, validated configuration and self-monitoring.
- I can design metric contracts using correct types, units, names and bounded labels.
- I can estimate and measure series cardinality before production rollout.
- I can configure static/dynamic discovery, relabel targets and filter samples safely.
- I can operate node exporter, blackbox exporter and a tightly controlled Pushgateway workflow.
- I can write and explain PromQL for counters, ratios, histograms, joins, missing data and SLOs.
- I can create recording rules with deterministic tests and measured performance.
- I can design actionable alerts and validate pending, firing, recovery and missing-data behavior.
- I can configure Alertmanager routing, grouping, inhibition, silences, templates, receivers and HA.
- I can perform controlled failure, recovery, rollback, capacity and security tests.
- I can package sanitized evidence and explain trade-offs in an interview.

Ten Additional Practice Projects

1. Cardinality Laboratory

Generate bounded and unbounded labels, measure series growth, then implement instrumentation and ingestion guardrails.

2. Linux Capacity Workbook

Create CPU, memory, filesystem and network recording rules plus capacity alerts with documented exclusions.

3. Black-Box Probe Matrix

Probe HTTP, TCP, DNS and TLS scenarios; compare phase duration and failure classification.

4. Batch Reliability Monitor

Instrument three short-lived jobs, manage Pushgateway cleanup and detect stale success metrics.

5. PromQL Test Suite

Build deterministic tests for resets, missing data, vector matching, histograms and zero denominators.

6. SLO Burn-Rate Pack

Create fast/slow burn rules for two objectives and validate against synthetic request logs.

7. Alert Quality Review

Classify 30 sample alerts into page, ticket, dashboard or remove; provide response contracts.

8. Alertmanager Route Simulator

Test a multi-team routing tree, grouping, continue behavior, inhibition and silence matching.

9. Monitoring-Platform Self-Monitoring

Alert on scrape gaps, rule failures, TSDB pressure, query latency and notification failures.

10. HA Failure Exercise

Run paired Prometheus and Alertmanager instances through node, network and receiver failures.

30-Day Prometheus Practice Plan

Period	Focus
Days 1-3	Deploy Prometheus, inspect TSDB/runtime/self-metrics, test restart and rollback.

Period	Focus
Days 4-6	Instrument an HTTP service; validate names, types, units, labels and cardinality.
Days 7-9	Build scrape jobs, service discovery examples, relabeling and sample limits.
Days 10-12	Operate node exporter, blackbox exporter and a short-lived Pushgateway job.
Days 13-16	Practice selectors, rates, aggregation, ratios, offsets and missing-data queries.
Days 17-20	Practice vector matching, histograms, SLO compliance and burn rates.
Days 21-23	Create recording rules, promtool tests and performance comparisons.
Days 24-26	Create alert rules, state-transition tests and runbooks.
Days 27-28	Build Alertmanager routes, grouping, inhibition, silences and HA tests.
Days 29-30	Complete capstone failure drills, evidence package and interview review.

Glossary

Term	Definition
Alert fingerprint	Identity derived from an alert label set; changing labels can create a different alert instance.
Cardinality	Number of unique time series created by metric names and label-value combinations.
Counter reset	A decrease in a cumulative counter, commonly caused by process restart.
Exporter	Service that translates another system's state into Prometheus metrics.
Exposition format	Text or protocol representation returned by a metrics endpoint.
External label	Label added to data/alerts to identify a Prometheus server or environment in broader systems.
Gauge	Metric representing a current value that can increase or decrease.
Histogram	Metric that counts observations in cumulative buckets and exposes sum/count.
Inhibition	Automatic Alertmanager suppression of target alerts while a matching source alert is active.
Instant vector	Set of time series with one sample per series at an evaluation time.
Metric relabeling	Transformation or filtering of samples after scrape and before

Term	Definition
	ingestion.
PromQL	Prometheus Query Language for selecting and transforming time series.
Range vector	Set of time series with a time window of samples per series.
Recording rule	Periodic expression whose result is stored as a new time series.
Relabeling	Rule-based label transformation/filtering used in discovery, ingestion or alert delivery stages.
Scrape	Prometheus collection request to a target metrics endpoint.
Silence	Time-bounded Alertmanager matcher that suppresses matching notifications.
TSDB	Time-series database used by Prometheus for local sample storage.

Official Documentation Index

- Prometheus documentation: <https://prometheus.io/docs/introduction/overview/>
- Getting started: https://prometheus.io/docs/prometheus/latest/getting_started/
- Prometheus configuration: <https://prometheus.io/docs/prometheus/latest/configuration/configuration/>
- PromQL basics: <https://prometheus.io/docs/prometheus/latest/querying/basics/>
- PromQL operators: <https://prometheus.io/docs/prometheus/latest/querying/operators/>
- PromQL functions: <https://prometheus.io/docs/prometheus/latest/querying/functions/>
- Metric types: https://prometheus.io/docs/concepts/metric_types/
- Instrumentation practices: <https://prometheus.io/docs/practices/instrumentation/>
- Metric and label naming: <https://prometheus.io/docs/practices/naming/>
- Histograms and summaries: <https://prometheus.io/docs/practices/histograms/>
- Recording rules: https://prometheus.io/docs/prometheus/latest/configuration/recording_rules/
- Alerting rules: https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/
- Rule unit testing: https://prometheus.io/docs/prometheus/latest/configuration/unit_testing_rules/
- Alertmanager overview: <https://prometheus.io/docs/alerting/latest/alertmanager/>
- Alertmanager configuration: <https://prometheus.io/docs/alerting/latest/configuration/>
- Alertmanager high availability: https://prometheus.io/docs/alerting/latest/high_availability/
- Writing exporters: https://prometheus.io/docs/instrumenting/writing_exporters/
- Node exporter guide: <https://prometheus.io/docs/guides/node-exporter/>
- Blackbox exporter repository: https://github.com/prometheus/blackbox_exporter
- Pushgateway documentation: <https://github.com/prometheus/pushgateway>

Version Note Prometheus and related components evolve. Verify current stable releases, migration guidance, feature flags, configuration schemas and security recommendations in official documentation before production implementation.

VOLUME 11B COMPLETE

Prometheus Engineering, PromQL, Exporters and Alertmanager

Next Part Volume 11C - Grafana, Centralized Logging, Loki and Elastic/OpenSearch Patterns.